



Nama : Triyana Dewi Fatmawati
NIM : 2241720206
Kelas : TI – 3D
Nomor : 21
Mata Kuliah : Big Data

Tugas 14 – Unsupervised Learning dengan MLlib Spark

Langkah Praktikum:

1) Inisialisasi Spark Session

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("Unsupervised Learning with MLlib") \
    .config("spark.executor.memory", "4g") \
    .getOrCreate()
```

2) Persiapan Data

Dataset: Sample dari dataset Iris (sepal length, sepal width, petal length, petal width)

```
from pyspark.ml.linalg import Vectors

data = [
    (Vectors.dense([5.1, 3.5, 1.4, 0.2]),),
    (Vectors.dense([4.9, 3.0, 1.4, 0.2]),),
    (Vectors.dense([4.7, 3.2, 1.3, 0.2]),),
    (Vectors.dense([7.0, 3.2, 4.7, 1.4]),),
    (Vectors.dense([6.4, 3.2, 4.5, 1.5]),),
    (Vectors.dense([6.9, 3.1, 4.9, 1.5]),)
]

df = spark.createDataFrame(data, ["features"])
```

3) Feature Scaling

```
from pyspark.ml.feature import VectorAssembler, StandardScaler

scaler = StandardScaler(
    inputCol="features",
    outputCol="scaled_features",
    withStd=True,
    withMean=True)

scaled_df = scaler.fit(df).transform(df)
```

4) Pelatihan Model K-Means

```
from pyspark.ml.clustering import KMeans

kmeans = KMeans().setK(3).setSeed(1)
model = kmeans.fit(scaled_df)

# Prediksi cluster
predictions = model.transform(scaled_df)
predictions.show()
```

```
+-----+-----+-----+
|      features|    scaled_features|prediction|
+-----+-----+-----+
|[5.1,3.5,1.4,0.2]|[-0.6983599143217...|         1|
|[4.9,3.0,1.4,0.2]|[-0.8888217091367...|         1|
|[4.7,3.2,1.3,0.2]|[-1.0792835039517...|         1|
|[7.0,3.2,4.7,1.4]|[1.11102713642096...|         0|
|[6.4,3.2,4.5,1.5]|[0.53964175197589...|         2|
|[6.9,3.1,4.9,1.5]|[1.01579623901345...|         0|
+-----+-----+-----+
```

5) Evaluasi Model

```
from pyspark.ml.evaluation import ClusteringEvaluator

evaluator = ClusteringEvaluator()
silhouette = evaluator.evaluate(predictions)
print("Silhouette with squared euclidean distance = " + str(silhouette))

# Within Set Sum of Squared Errors

# Dengan ini:
wsse = model.summary.trainingCost
print("Within Set Sum of Squared Errors = " + str(wsse))
```

```
Silhouette with squared euclidean distance = 0.7692979876379087
Within Set Sum of Squared Errors = 0.2483333333333331
```

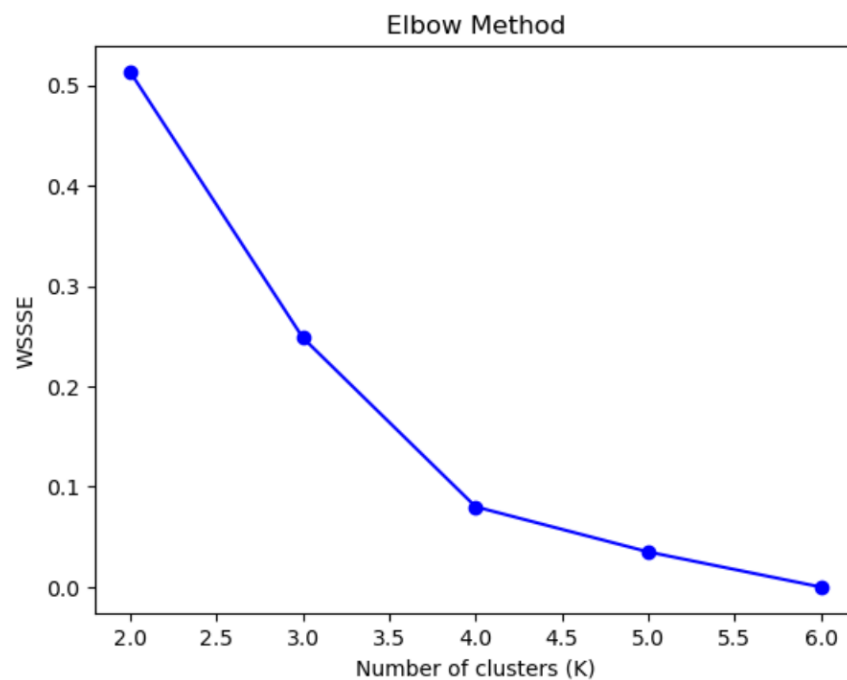
6) Menentukan Jumlah Cluster Optimal (Elbow Method)

```
import matplotlib.pyplot as plt

costs = []
for k in range(2, 7):
    kmeans = KMeans().setK(k).setSeed(1)
    model = kmeans.fit(scaled_df)
    costs.append(model.summary.trainingCost) # Gunakan trainingCost
    print(f"K={k}, WSSSE={costs[-1]}")

# Visualisasi
import matplotlib.pyplot as plt
plt.plot(range(2, 7), costs, 'bo-')
plt.xlabel('Number of clusters (K)')
plt.ylabel('WSSSE')
plt.title('Elbow Method')
plt.show()
```

K=2, WSSSE=0.5133333333333331
 K=3, WSSSE=0.24833333333333313
 K=4, WSSSE=0.08000000000000007
 K=5, WSSSE=0.03500000000000002
 K=6, WSSSE=0.0



7) Dimensionality Reduction dengan PCA

```
from pyspark.ml.feature import PCA

pca = PCA(k=2, inputCol="features", outputCol="pca_features")
pca_model = pca.fit(scaled_df)

# Transform data
pca_result = pca_model.transform(scaled_df)
pca_result.select("pca_features").show(truncate=False)
```

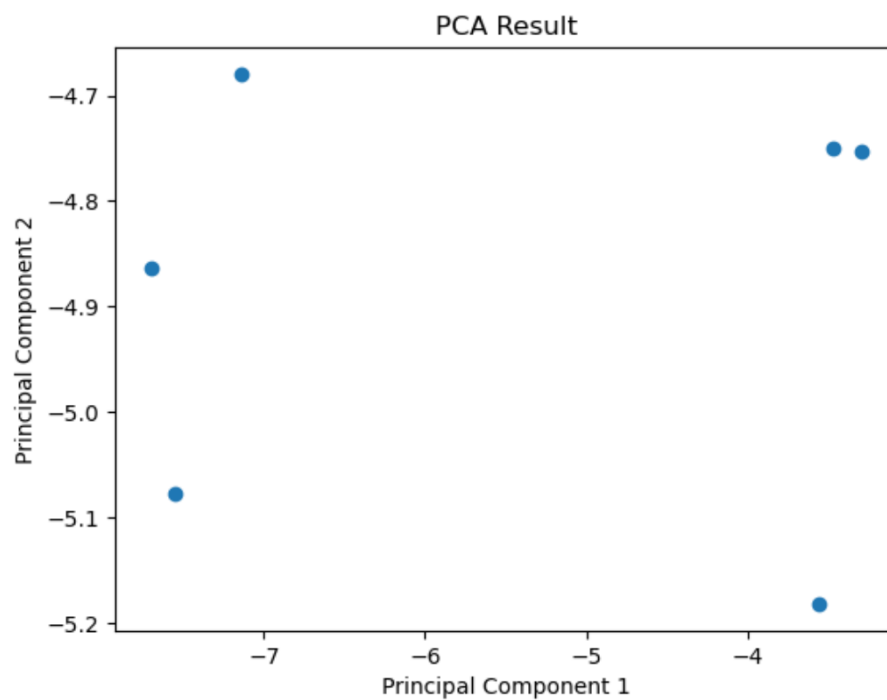
```
+-----+
|pca_features|
+-----+
|[-3.5540691264840647, -5.181690130080339]|
|[-3.4683844688148193, -4.75023569826665]|
|[-3.288720840381228, -4.753062654835084]|
|[-7.550562728189457, -5.077394269765556]|
|[-7.1350916994969475, -4.679675606096408]|
|[-7.701700199921558, -4.863751494788255]|
+-----+
```

8) Visualisasi Hasil PCA

```
import numpy as np

# Ekstrak komponen utama
pca_data = pca_result.select("pca_features").rdd.map(lambda x: x[0]).collect()
x = [float(d[0]) for d in pca_data]
y = [float(d[1]) for d in pca_data]

plt.scatter(x, y)
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('PCA Result')
plt.show()
```



9) Integrasi PCA dan K-Means

Clustering pada Data yang Direduksi

```
kmeans_pca = KMeans().setK(3).setSeed(1).setFeaturesCol("pca_features")
pca_kmeans_model = kmeans_pca.fit(pca_result)

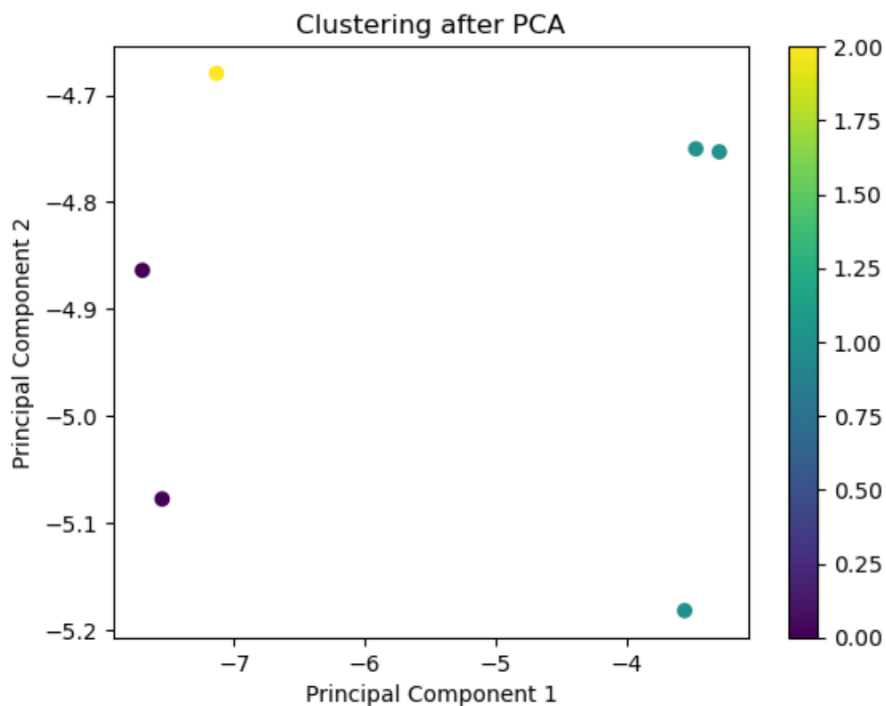
# Prediksi cluster
pca_predictions = pca_kmeans_model.transform(pca_result)
pca_predictions.select("pca_features", "prediction").show()
```

```
+-----+-----+
|      pca_features|prediction|
+-----+-----+
| [-3.5540691264840...|          1|
| [-3.4683844688148...|          1|
| [-3.2887208403812...|          1|
| [-7.5505627281894...|          0|
| [-7.1350916994969...|          2|
| [-7.7017001999215...|          0|
+-----+-----+
```

10) Visualisasi Cluster Hasil PCA

```
# Gabungkan hasil clustering dengan data PCA
cluster_data = pca_predictions.select("pca_features", "prediction").collect()
x = [float(d[0][0]) for d in cluster_data]
y = [float(d[0][1]) for d in cluster_data]
clusters = [int(d[1]) for d in cluster_data]

plt.scatter(x, y, c=clusters)
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('Clustering after PCA')
plt.colorbar()
plt.show()
```



Kesimpulan

- Unsupervised learning berguna untuk eksplorasi data tanpa label.
- K-Means untuk clustering, PCA untuk reduksi dimensi.
- Evaluasi clustering dengan WCSS & Silhouette Score.
- PCA + K-Means mempercepat komputasi & memudahkan visualisasi.

Aplikasi di Dunia Nyata

- Segmentasi pelanggan (marketing).
- Deteksi anomali (fraud detection).
- Kompresi gambar